

TECH GLOSSARY HUB

(FLASK VERSION)

Project Documentation

1. Introduction

Tech Glossary Hub — Flask Version is a Flask-based technical glossary web application designed to organize and present technical terminology in a structured, searchable, and developer-friendly interface. The project focuses on backend web development using Flask, SQLAlchemy, SQLite, Jinja templates, HTML, CSS, and JavaScript.

2. Project Overview

The application provides a centralized glossary containing 250 technical terms organized across 12 categories. Each glossary term includes a name, slug, definition, example, category, and difficulty level.

- Framework: Flask
- Database: SQLite
- ORM: Flask-SQLAlchemy / SQLAlchemy
- Templates: Jinja2
- Frontend: HTML, CSS, JavaScript
- Categories: 12
- Glossary Terms: 250
- Difficulty Levels: Beginner, Moderate, Advanced
- Deployment: Render

3. Objectives

- Build a practical technical glossary using Flask.
- Organize technical concepts into clearly defined categories.
- Provide search and category-based glossary navigation.
- Create individual detail pages for glossary terms.
- Practice Flask routing, templates, SQLAlchemy models, and database queries.
- Implement a responsive and accessible developer-oriented interface.
- Deploy the Flask application to Render.

4. Features

- Home page with project overview and glossary statistics.
- 12 technical categories.
- 250 glossary terms.
- Glossary search functionality.
- Category filtering.
- Pagination for glossary results.
- Individual term detail pages.

- Beginner, Moderate, and Advanced level indicators.
- Previous and next navigation between terms.
- Related terms section.
- Versions page linking the different Tech Glossary Hub implementations.
- About page.
- Contact page.
- Responsive layout.
- Accessibility foundation including skip navigation and visible focus states.
- Dark developer-oriented visual design.
- Footer containing project, portfolio, GitHub, LinkedIn, and email links.
- Render deployment support.

5. System Architecture

The application follows a lightweight Flask web architecture. Incoming HTTP requests are handled by Flask routes. The routes query SQLAlchemy models, obtain the required database records, and render Jinja2 templates. Static CSS and JavaScript files provide the presentation and client-side behavior.

- Client requests a page through a browser.
- Flask receives the HTTP request.
- The appropriate route in the application blueprint is selected.
- The route queries the database through SQLAlchemy models.
- The requested data is passed to a Jinja2 template.
- The template generates the HTML response.
- CSS and JavaScript provide styling, responsive behavior, and client-side interactions.
- The response is returned to the browser.

High-level flow:

Browser → Flask Application → Routes / Blueprint → SQLAlchemy Models → SQLite Database → Jinja2 Templates → HTML/CSS/JavaScript → Browser

6. Technologies Used

Technology	Purpose	Usage
Python	Programming language	Application and backend logic
Flask	Web framework	Routing and application development
Flask-SQLAlchemy	ORM integration	Database interaction
SQLAlchemy	ORM	Models and database queries
SQLite	Database	Glossary and category storage
Jinja2	Template engine	Dynamic HTML rendering
HTML5	Markup	Page structure
CSS3	Styling	Responsive UI and visual design
JavaScript	Client-side scripting	Frontend interactions
Gunicorn	WSGI server	Production application server
Render	Deployment platform	Hosting the Flask application
Git & GitHub	Version control	Source code management

7. Project Structure

```
tech-glossary-hub-flask/
├── app/
│   ├── models/
│   │   ├── category.py
│   │   └── glossary_term.py
│   ├── routes/
│   │   └── main.py
│   ├── templates/
│   │   ├── base.html
│   │   ├── home.html
│   │   ├── categories.html
│   │   ├── category_detail.html
│   │   ├── glossary.html
│   │   ├── term_detail.html
│   │   ├── versions.html
│   │   ├── contact.html
│   │   ├── about.html
│   │   └── components/
│   │       ├── navbar.html
│   │       └── footer.html
│   ├── static/
│   │   ├── css/
│   │   │   ├── style.css
│   │   │   └── components.css
│   │   └── js/
│   │       └── main.js
├── instance/
│   └── glossary.db
├── config.py
├── run.py
├── seed.py
├── check_database.py
├── requirements.txt
├── .gitignore
└── README.md
```

8. Database Design

The application uses SQLite with SQLAlchemy models. The main data entities are Category and GlossaryTerm. A category can contain multiple glossary terms, while each glossary term belongs to one category.

Entity	Field	Type	Description
Category	id	Integer	Primary key
Category	name	String	Category name
Category	slug	String	URL-friendly unique identifier
GlossaryTerm	id	Integer	Primary key
GlossaryTerm	name	String	Technical term name
GlossaryTerm	slug	String	URL-friendly unique

			identifier
GlossaryTerm	definition	Text	Technical definition
GlossaryTerm	example	Text	Example or usage explanation
GlossaryTerm	category_id	Integer	Foreign key to Category
GlossaryTerm	level	String	Beginner, Moderate, or Advanced
GlossaryTerm	created_at	DateTime	Record creation timestamp

Relationship:

Category 1 —————< GlossaryTerm

Categories included:

- Flask & Microframeworks
- REST APIs
- Backend Architecture
- Web Authentication
- Databases & ORM
- Web Security
- Web Performance
- Testing & Debugging
- Background Processing
- Web Deployment
- Software Architecture
- Python Web Ecosystem

9. Project Workflow

- Create the Flask application and configuration.
- Initialize the database and SQLAlchemy integration.
- Create Category and GlossaryTerm models.
- Create the main Flask blueprint and application routes.
- Build the base template, navigation, and footer components.
- Develop category and glossary pages.
- Implement search, filtering, and pagination.
- Develop individual term detail pages and related-term navigation.
- Add Versions, About, and Contact pages.
- Improve responsive behavior and accessibility.
- Prepare the glossary dataset containing 250 terms.
- Verify database counts and application behavior.
- Configure Gunicorn and Render deployment.
- Verify the deployed application.

10. Implementation Details

10.1 Application Factory

The Flask application uses an application factory pattern through `create_app()`. This separates application initialization from the executable entry point and allows the application to be created in different environments.

10.2 Routing

The main blueprint provides the following routes:

- /
- /categories
- /category/<slug>
- /glossary
- /term/<slug>
- /versions
- /contact
- /about

10.3 Glossary Search and Filtering

The glossary page accepts a search query and category slug. Searches can match the term name, definition, or example. Results are ordered alphabetically and displayed through pagination.

10.4 Term Details

Each term has a dedicated URL based on its unique slug. The detail page displays the definition, example, category, difficulty level, related terms, and previous/next navigation within the category.

10.5 Database Seeding

The seed script is used to populate the application with the predefined categories and glossary dataset. A database verification script is also available to check category, term, and difficulty-level counts.

10.6 UI and Accessibility

The interface uses a dark developer-oriented design with reusable navigation and footer components. Accessibility improvements include a skip-to-content link, visible keyboard focus states, semantic page structure, visually hidden helper content, and reduced-motion support.

10.7 Deployment

The Flask application is configured for deployment on Render using Gunicorn as the production WSGI server. The project dependencies are declared in `requirements.txt`.

11. Learning Outcomes

- Understanding Flask application structure and routing.
- Working with application factories and blueprints.
- Creating database models with SQLAlchemy.
- Building one-to-many database relationships.
- Using Jinja2 templates for dynamic web pages.

- Implementing search, filtering, and pagination.
- Designing reusable template components.
- Organizing static CSS and JavaScript assets.
- Implementing basic accessibility practices.
- Preparing a Flask application for production deployment.
- Using Git and GitHub for project version control.
- Deploying a Python web application using Render.

12. Conclusion

Tech Glossary Hub — Flask Version demonstrates the development of a complete Flask web application from project structure and database modeling through frontend presentation, search functionality, accessibility improvements, and deployment. The project provides a practical foundation for understanding how Flask, SQLAlchemy, Jinja2, SQLite, and production deployment tools work together in a Python web application.

13. Project Links & Resources

- **GitHub Repository:** <https://github.com/charanepuri/tech-glossary-hub-flask>
- **Live Demo:** <https://tech-glossary-hub-flask.onrender.com/>

14. Author

Charan Teja EPURI

Aspiring Python Full Stack Developer

- ❖ **Django Portfolio:** <https://portfolio-site-django.onrender.com>
- ❖ **GitHub:** <https://github.com/charanepuri>
- ❖ **React Portfolio:** <https://charan-react-portfolio.vercel.app>
- ❖ **LinkedIn:** <https://www.linkedin.com/in/charan-teja-972aa9231>
- ❖ **Flask Portfolio:** <https://flask-developer-dashboard-portfolio.onrender.com/>